🗂️ Kanban Task Board: TypeScript Learning Project

Goal: learn TypeScript with React by building a small but real app.
Stack: React + TypeScript + Vite + Tailwind CSS. No backend; data is saved in Local Storage.
Time: about 7 days at 1 to 3 hours a day.

> Golden rules
>
> - `"strict": true` stays on. Never turn it off.
> - No `any`. If you're stuck, use `unknown` and narrow it, or ask for help.
> - Read every TypeScript error slowly. Every error is a lesson.

---

1. Features

Must have (MVP)

- 3 columns: **To Do**, **In Progress**, **Done**
- Add a task: title, description (optional), priority (low / medium / high), due date (optional)
- Edit a task and delete a task
- Move tasks between columns with **drag & drop** (native HTML5 drag events, no library)
- Task counter on each column
- Data is saved in Local Storage and survives a refresh
- Responsive: columns stack vertically on mobile

Nice to have (after the MVP)

- Search tasks by title
- Filter by priority
- Sort by due date or priority
- "Overdue" badge when the due date has passed
- Light / dark mode toggle
- Undo the last delete (toast with an "Undo" button)

---

2. Core types (`src/types/task.ts`)

Write these first. They are the heart of the project.

```
export type Status = "todo" | "in-progress" | "done";
export type Priority = "low" | "medium" | "high";

export interface Task {
  id: string;
  title: string;
  description?: string;      // optional
  priority: Priority;
  status: Status;
  dueDate?: string;          // ISO date string, optional
  createdAt: string;
}

// The data the form sends: no id, status or createdAt yet
export type NewTask = Omit<Task, "id" | "status" | "createdAt">;

// For editing: everything optional except the id
export type TaskUpdate = Partial<NewTask> & { id: string };

export interface Column {
  id: Status;
  title: string;
}
```

Reducer actions (discriminated union)

```
export type TaskAction =
  | { type: "ADD_TASK"; payload: NewTask }
  | { type: "UPDATE_TASK"; payload: TaskUpdate }
  | { type: "DELETE_TASK"; payload: { id: string } }
  | { type: "MOVE_TASK"; payload: { id: string; status: Status } };
```

Inside the reducer, TypeScript knows the exact `payload` type for each `case`. This is the most important pattern you'll learn here.

---

3. Folder structure

```
src/
├── types/
│   └── task.ts              # all types live here
├── constants/
│   └── columns.ts           # the 3 columns + priority labels/colors
├── hooks/
│   ├── useLocalStorage.ts   # generic hook: useLocalStorage<T>()
│   └── useTasks.ts          # useReducer + Local Storage combined
├── reducers/
│   └── tasksReducer.ts      # (state: Task[], action: TaskAction) =>
Task[]
├── components/
│   ├── Board.tsx            # renders the 3 columns
│   ├── Column.tsx           # one column + drop zone
│   ├── TaskCard.tsx         # one draggable card
│   ├── TaskForm.tsx         # add/edit form in a modal
│   ├── PriorityBadge.tsx
│   └── Modal.tsx
├── utils/
│   └── date.ts              # isOverdue(), formatDate()
├── App.tsx
└── main.tsx
```

---

4. Day-by-day plan

Day 1: Setup and types

```
npm create vite@latest kanban-board -- --template react-ts
cd kanban-board
npm install
```

- Install Tailwind (follow the official Vite guide).
- Write `src/types/task.ts` exactly as above.
- Write `src/constants/columns.ts` with a typed array: `export const COLUMNS: Column[] = [...]`.

You'll learn: type aliases, interfaces, union types, optional properties, `Omit` and `Partial`.

Day 2: Static UI with typed props

- Build `Board`, `Column` and `TaskCard` with **hardcoded sample tasks** (a `Task[]` array).

- Type every component's props:

```
interface TaskCardProps {
  task: Task;
  onEdit: (task: Task) => void;
  onDelete: (id: string) => void;
}
```

- Build `PriorityBadge` with a `Record<Priority, string>` for colors:

```
const PRIORITY_STYLES: Record<Priority, string> = {
  low: "bg-green-500/15 text-green-400",
  medium: "bg-yellow-500/15 text-yellow-400",
  high: "bg-red-500/15 text-red-400",
};
```

You'll learn: typing props, function types, `Record<K, V>`, and how TS catches a missing priority.

Day 3: State with useReducer

- Write `tasksReducer` with a `switch` on `action.type`.
- Use `crypto.randomUUID()` for ids and `new Date().toISOString()` for `createdAt`.
- Add an exhaustive check at the end of the switch:

```
default: {
  const _exhaustive: never = action;
  return state;
}
```

  If you add a new action type later and forget to handle it, TypeScript will show an error here.

You'll learn: discriminated unions, narrowing, the `never` type, immutable updates with types.

Day 4: The form

- Build `TaskForm` for both **add** and **edit** modes.
- Type the events:

```
const handleChange = (
  e: React.ChangeEvent<HTMLInputElement | HTMLTextAreaElement |
HTMLSelectElement>
```

```
) => { ... };

const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => { ... };
```

- Validate: title is required, and show an error message if it's empty.
- Build a reusable `Modal` that accepts `children: React.ReactNode`.

You'll learn: event types, `React.ReactNode`, controlled inputs with typed state.

Day 5: Drag & drop

- On `TaskCard`: `draggable`, and `onDragStart` saves the task id:

```
const handleDragStart = (e: React.DragEvent<HTMLDivElement>) => {
  e.dataTransfer.setData("text/plain", task.id);
};
```

- On `Column`: `onDragOver` (call `e.preventDefault()`) and `onDrop`, which dispatches `MOVE_TASK`.
- Highlight the column while a card is dragged over it.

You'll learn: `React.DragEvent`, typing DOM events, passing typed callbacks down.

Day 6: Generic Local Storage hook

```
function useLocalStorage<T>(key: string, initialValue: T) {
  const [value, setValue] = useState<T>(() => {
    try {
      const stored = localStorage.getItem(key);
      return stored ? (JSON.parse(stored) as T) : initialValue;
    } catch {
      return initialValue;
    }
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue] as const;
}
```

- Combine it with the reducer inside `useTasks()`, which returns `{ tasks, addTask, updateTask, deleteTask, moveTask }`.
- Write `isOverdue(task: Task): boolean` in `utils/date.ts` and show an "Overdue" badge.

You'll learn: generics `<T>`, `as const`, typing a custom hook's return value.

Day 7: Polish, deploy, README

- Responsive check at 375px, 768px and 1440px.
- Empty states ("No tasks yet").
- Apply your brand: navy `#101828` background, indigo `#4F39F6` accent, Poppins font.
- `npm run build` must pass with **zero** TypeScript errors.
- Push to GitHub (`kanban-board`), connect to Netlify, add `preview.png` and Open Graph tags.
- Write the README the same way as your other projects, and add a section called **"What I learned about TypeScript"**. Recruiters love this.

---

5. TypeScript checklist

By the end you should be comfortable with:

- [ ] `type` vs `interface`
- [ ] Union types and literal types (`"low" | "medium" | "high"`)
- [ ] Optional properties (`description?`)
- [ ] Utility types: `Omit`, `Partial`, `Record`
- [ ] Typing component props and callbacks
- [ ] Typing events: `ChangeEvent`, `FormEvent`, `DragEvent`
- [ ] Discriminated unions with `useReducer`
- [ ] Exhaustive checks with `never`
- [ ] Generics in a custom hook (`useLocalStorage<T>`)
- [ ] `as const` and typed return values

---

6. After this project

Move to the **real-time chat app with TypeScript + Firebase**. You'll reuse:

- interfaces for your data (`User`, `Message`, `ChatRoom`)
- discriminated unions for app state
- generic hooks (`useCollection<T>()`)

📚 References

- React TypeScript Cheatsheet
- TypeScript Handbook (official docs)